



**COLLEGIUM
WITELONA**

Uczelnia Państwowa



Dokumentacja techniczna
MultiRental

Platforma marketplace do wypożyczania przedmiotów

Autorzy:

Maciej Wróblewski

Paweł Miedziński

Maksymilian Wojtkowski

s2PAM1(1)

Spis treści

I. Koncepcja – 3

1. Cel projektu bazy danych – 3
2. Opis dziedziny przedmiotowej – 3
3. Założenia wstępne – 4

II. Specyfikacja wymagań systemu – 5

1. Użytkownicy systemu oraz ich role – 5
2. Wymagania funkcjonalne – 5
3. Wymagania niefunkcjonalne – 7

III. Model bazy danych – 8

1. Schemat bazy danych – 8
2. Opis poszczególnych encji – 8
3. Model MVC – 11

IV. Interfejs użytkownika – 12

1. Opis GUI – 12
2. Implementacja – 15
3. Wykorzystanie systemu kontroli wersji – 18

Bibliografia – 19

I. Koncepcja

1. Cel projektu bazy danych

Celem projektu było zaprojektowanie oraz implementacja systemu bazodanowego wspierającego działanie internetowej platformy umożliwiającej wypożyczanie przedmiotów pomiędzy użytkownikami.

Projekt „MultiRental” został stworzony jako aplikacja webowa typu marketplace, pozwalająca użytkownikom publikować własne oferty wynajmu, wyszukiwać dostępne przedmioty, realizować proces wypożyczenia oraz komunikować się między sobą za pomocą systemu prywatnych wiadomości.

Istotnym elementem projektu było również zaprojektowanie odpowiedniej struktury bazy danych zapewniającej integralność danych oraz poprawne relacje pomiędzy encjami systemu.

W aplikacji zaimplementowano funkcjonalności obejmujące między innymi zarządzanie ofertami, obsługę wypożyczeń, system ocen użytkowników, prywatne wiadomości oraz panel administracyjny umożliwiający nadzorowanie systemu.

Projekt został wykonany przy wykorzystaniu frameworka Laravel, relacyjnej bazy danych MySQL oraz środowiska Docker umożliwiającego łatwe uruchamianie aplikacji.

2. Opis dziedziny przedmiotowej

Projekt dotyczy dziedziny internetowych platform marketplace umożliwiających wynajem przedmiotów pomiędzy użytkownikami.

W praktyce wiele osób posiada przedmioty używane sporadycznie, takie jak sprzęt elektroniczny, aparaty fotograficzne, konsole do gier, narzędzia lub inne urządzenia, które przez większość czasu pozostają niewykorzystywane. Jednocześnie inni użytkownicy potrzebują dostępu do takich zasobów jedynie na określony czas.

System MultiRental został zaprojektowany w celu umożliwienia bezpiecznego procesu publikowania ofert, wyszukiwania przedmiotów, realizowania wypożyczeń oraz budowania zaufania pomiędzy użytkownikami poprzez system ocen i recenzji.

Aplikacja umożliwia również komunikację prywatną pomiędzy użytkownikami oraz zapewnia funkcjonalności administracyjne związane z nadzorowaniem platformy.

3. Założenia wstępne

Do założeń projektu należą:

- System działa jako aplikacja webowa,
- Użytkownik może bezpiecznie utworzyć konto oraz logować się do systemu,
- Użytkownik może dodawać własne oferty wypożyczeń,
- Każdy przedmiot posiada właściciela,
- Użytkownik może łatwo wyszukiwać oraz filtrować oferty,
- Użytkownik może wypożyczać jedynie dostępne przedmioty,
- Użytkownik może wystawiać oceny i recenzować wynajmujących,
- Platforma posiada system prywatnych wiadomości pomiędzy użytkownikami,
- Administrator posiada dodatkowe uprawnienia, służące do moderowania aplikacji,
- System wykorzystuje bazę danych MySQL,
- Aplikacja uruchamiana jest z wykorzystaniem środowiska Docker.

II. Specyfikacja wymagań systemu

1. Użytkownicy systemu oraz ich role

W platformie MultiRental wyróżniono trzy podstawowe role użytkowników, różniące się zakresem dostępnych funkcjonalności.

(a) Gość (użytkownik niezalogowany)

Użytkownik niezalogowany może przeglądać publicznie dostępne zasoby systemu. Ma możliwość przeglądania ofert przedmiotów dostępnych na platformie, jednak nie posiada uprawnień do korzystania z funkcji wymagających autoryzacji.

(b) Użytkownik

Podstawową rolą systemu jest użytkownik posiadający konto. Po zalogowaniu użytkownik uzyskuje dostęp do pełnej funkcjonalności platformy.

Może on:

- tworzyć własne ogłoszenia wynajmu,
- edytować i usuwać własne przedmioty,
- przeglądać oraz filtrować dostępne oferty,
- realizować proces wypożyczenia,
- prowadzić prywatne rozmowy z innymi użytkownikami,
- wystawiać recenzje i oceny,
- zarządzać własnym profilem,
- dodawać zdjęcie profilowe oraz baner użytkownika.

(c) Administrator

Administrator posiada większe uprawnienia umożliwiające moderację systemu.

Do jego obowiązków należą:

- zarządzanie użytkownikami,
- usuwanie ofert naruszających zasady platformy,
- blokowanie użytkowników,
- monitorowanie poprawności działania systemu.

2. Wymagania funkcjonalne

W systemie MultiRental zaimplementowano następujące wymagania funkcjonalne:

1. Rejestracja użytkownika
2. Logowanie użytkownika
3. Edycja danych profilu
4. Dodawanie zdjęcia profilowego
5. Dodawanie banera

6. Tworzenie nowych ofert wynajmu
7. Edycja ofert
8. Usuwanie ofert
9. Wyświetlanie i przeglądanie ofert innych użytkowników
10. Wyszukiwanie przedmiotów
11. Filtrowanie ofert według dostępnych parametrów
12. Wypożyczanie przedmiotów
13. Zwrot przedmiotów
14. Automatyczne obliczanie ceny za wypożyczenie przedmiotu
15. Umieszczanie ocen oraz recenzji
16. Wyświetlanie średniej ocen użytkownika
17. System wiadomości prywatnych pomiędzy użytkownikami
18. Panel administratora
19. Moderacja użytkowników
20. Usuwanie ofert przez administratora
21. Banowanie/blokowanie użytkowników przez administratora

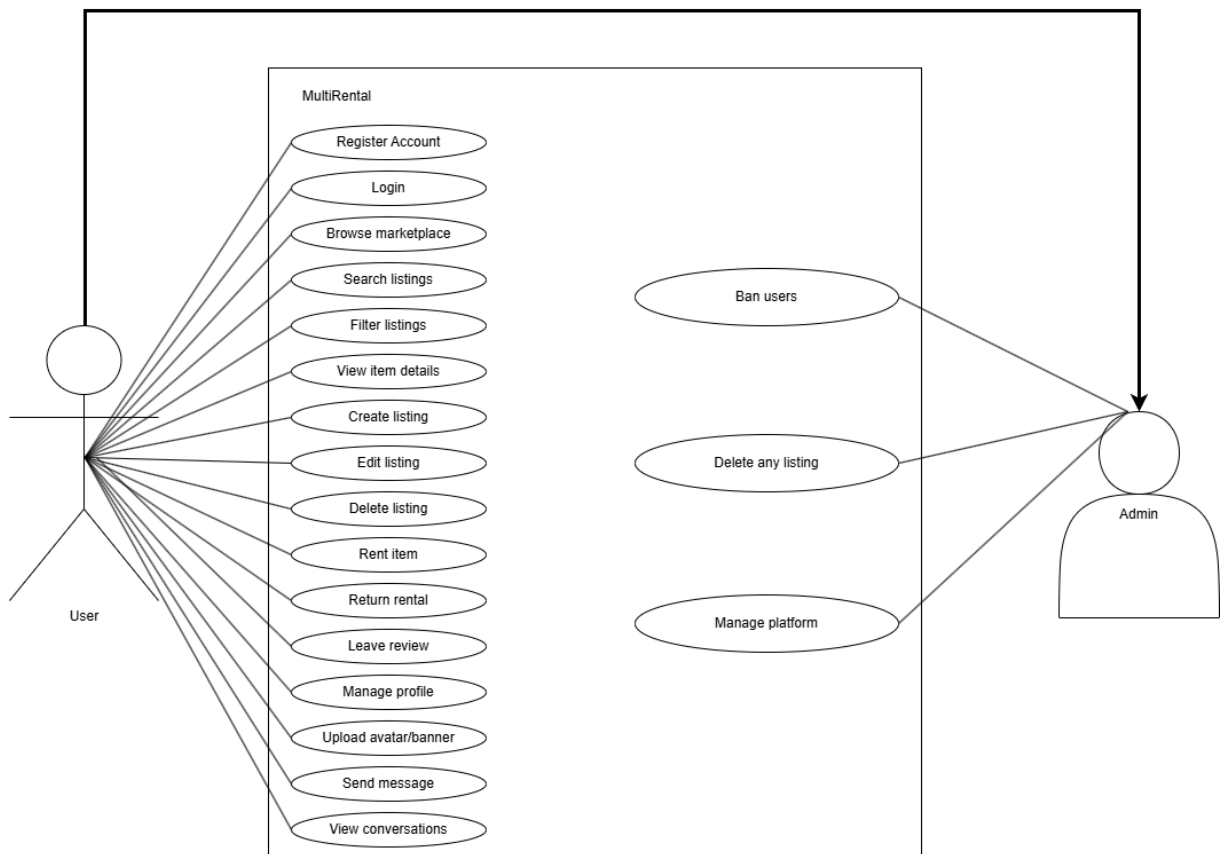


Diagram 1. Diagram use-case

3. Wymagania niefunkcjonalne

Ponadto, w systemie określono również wymagania niefunkcjonalne:

1. System powinien działać jako aplikacja webowa
2. Aplikacja powinna wykorzystywać relacyjną bazę danych MySQL
3. System powinien być uruchamialny przy wykorzystaniu środowiska Docker
4. Aplikacja powinna zapewniać integralność i nienaruszalność danych poprzez wykorzystanie relacji i kluczy obcych
5. System powinien zapewniać pełną autoryzację użytkowników
6. Aplikacja powinna być skalowalna i umożliwiać dalszą rozbudowę funkcjonalności
7. System powinien zapewniać poprawną obsługę wielu użytkowników
8. Interfejs użytkownika powinien być czytelny i intuicyjny
9. System powinien zapewniać odpowiedni poziom bezpieczeństwa danych

III. Model danych

1. Schemat bazy danych

Model danych systemu MultiRental został zaprojektowany w oparciu o relacyjną bazę danych MySQL. Struktura bazy danych opiera się na encjach reprezentujących główne elementy systemu, takie jak użytkownicy, przedmioty, wypożyczenia, wiadomości oraz recenzje.

Pomiędzy tabelami zastosowano relacje typu jeden-do-wielu oraz wiele-do-wielu, umożliwiające zachowanie integralności danych oraz poprawne odwzorowanie zależności biznesowych występujących w systemie.

W projekcie wykorzystano klucze główne oraz klucze obce zapewniające spójność danych i poprawność działania operacji wykonywanych przez aplikację.

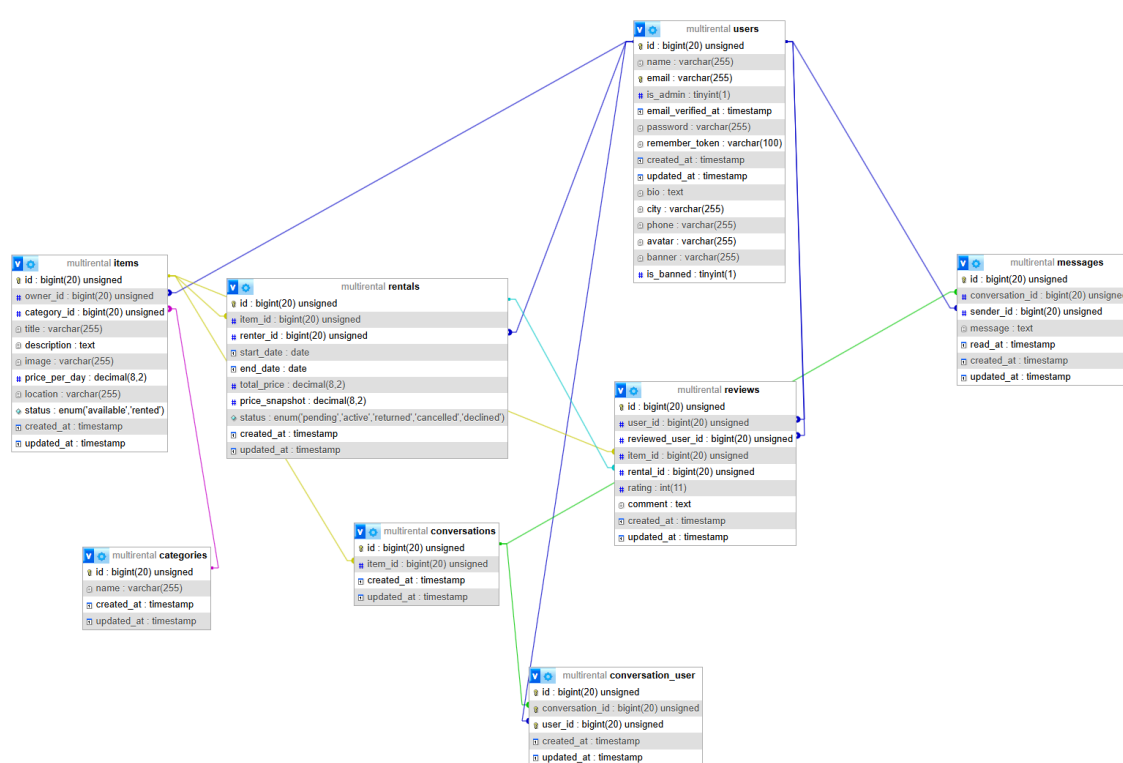


Diagram 2. Diagram ERD

2. Opis poszczególnych encji

1. users

Tabela users przechowuje kluczowe informacje dotyczące użytkowników systemu.

Najważniejsze pola:

- id – unikalny identyfikator użytkownika
- name – wyświetlana nazwa użytkownika
- email – unikalny adres poczty elektronicznej

- password – zahashowane hasło
- email_verified_at – data potwierdzenia adresu e-mail
- avatar – ścieżka do przechowywanego lokalnie zdjęcia profilowego
- banner – ścieżka do przechowywanego lokalnie banera profilu
- city – lokalizacja użytkownika
- phone – numer telefonu użytkownika
- bio – opis profilu użytkownika
- is_admin – flaga określająca uprawnienia administratora
- is_banned – status blokady użytkownika
- created_at – data utworzenia konta
- updated_at – data ostatniej modyfikacji

2. items

Tabela items przechowuje informacje dotyczące ofert wypożyczenia przedmiotów.

Najważniejsze pola:

- id – unikalny identyfikator przedmiotu
- owner_id – właściciel przedmiotu
- category_id – identyfikator kategorii przedmiotu
- title – nazwa przedmiotu
- description – opis przedmiotu
- location – lokalizacja
- image – zdjęcie przedmiotu
- price_per_day – cena za dzień wynajmu
- status – aktualny status przedmiotu (available lub rented)
- created_at – data utworzenia wpisu
- updated_at – data ostatniej edycji oferty

3. rentals

Tabela rentals odpowiada za proces wypożyczania przedmiotów.

Najważniejsze pola:

- id – unikalny identyfikator wypożyczenia
- renter_id – użytkownik wypożyczający
- item_id – wypożyczany przedmiot
- start_date – data rozpoczęcia okresu wypożyczenia
- end_date – data zakończenia okresu wypożyczenia
- total_price – całkowita cena wypożyczenia
- price_snapshot – zapis ceny przedmiotu w momencie wypożyczenia
- created_at – data utworzenia wypożyczenia
- updated_at – data ostatniej modyfikacji
- status – status wypożyczenia (active, returned, cancelled)

4. reviews

Tabela reviews przechowuje recenzje oraz oceny wystawiane użytkownikom po zakończonym procesie wypożyczenia.

Najważniejsze pola:

- id – unikalny identyfikator recenzji
- user_id – użytkownik wystawiający ocenę
- rental_id – identyfikator powiązanego z recenzją wypożyczenia
- item_id – identyfikator ocenianego przedmiotu
- reviewed_user_id – użytkownik otrzymujący ocenę
- rating – ocena użytkownika
- comment – treść recenzji
- created_at – data wytworzenia recenzji
- updated_at – data ostatniej modyfikacji

5. conversations

Tabela conversations odpowiada za przechowywanie konwersacji pomiędzy użytkownikami w systemie prywatnych wiadomości.

Najważniejsze pola:

- id – unikalny identyfikator konwersacji
- item_id – przedmiot, którego dotyczy rozmowa
- created_at – data utworzenia konwersacji
- updated_at – data ostatniej modyfikacji

6. conversation_user

Jest to tabela pośrednia odpowiadająca za relację wiele-do-wielu pomiędzy użytkownikami a konwersacjami systemu wiadomości prywatnych.

Najważniejsze pola:

- id – unikalny identyfikator wpisu
- conversation_id – identyfikator konwersacji
- user_id – identyfikator użytkownika uczestniczącego w konwersacji
- created_at – data utworzenia wpisu
- updated_at – data ostatniej modyfikacji

7. messages

Tabela messages przechowuje wiadomości wysyłane pomiędzy użytkownikami.

Najważniejsze pola:

- id – unikalny identyfikator wiadomości
- conversation_id – identyfikator konwersacji
- sender_id – identyfikator nadawcy wiadomości
- message – treść wiadomości

- read_at – data odczytania wiadomości
- created_at – data wysłania wiadomości
- updated_at – data ostatniej edycji wiadomości

8. categories

Tabela categories przechowuje kategorie przedmiotów dostępnych w systemie.

Najważniejsze pola:

- id – unikalny identyfikator kategorii
- name – nazwa kategorii
- created_at – data utworzenia kategorii
- updated_at – data ostatniej modyfikacji

3. Model MVC

Projekt został wykonany zgodnie z architekturą Model-View-Controller (MVC) wykorzystywaną przez framework Laravel.

Warstwa Model odpowiada za komunikację z bazą danych oraz logikę biznesową.

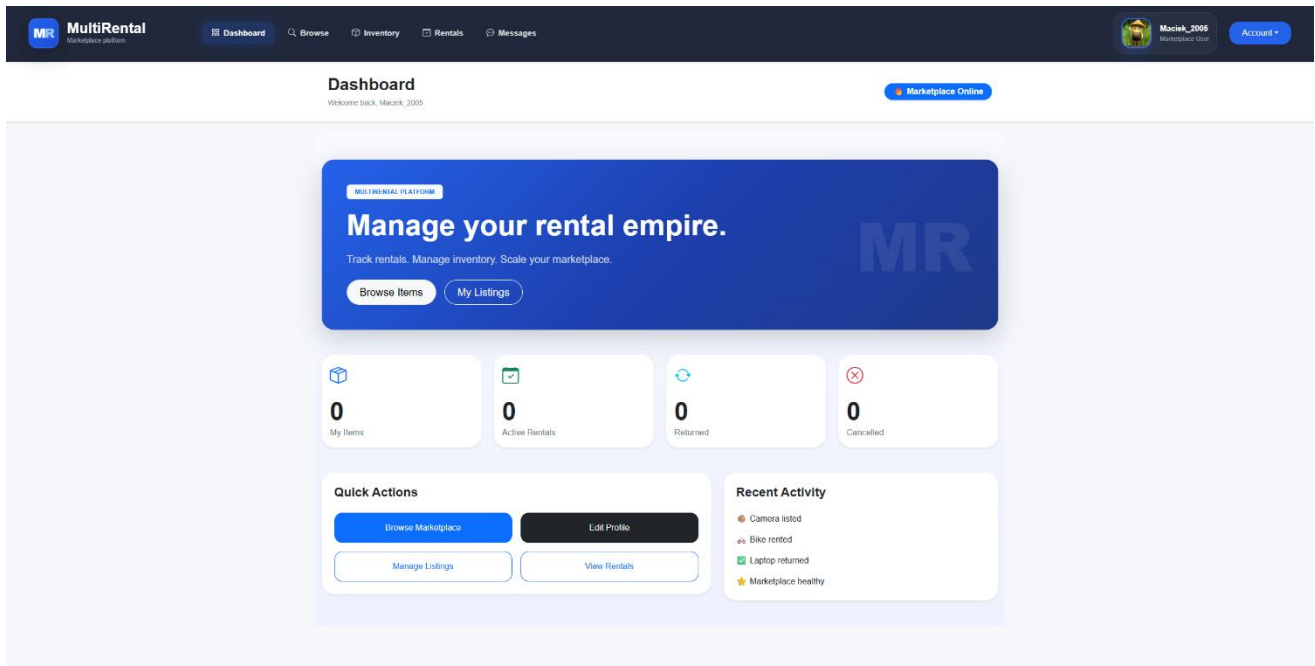
Warstwa View odpowiada za interfejs użytkownika realizowany przy użyciu szablonów Blade.

Warstwa Controller odpowiada za przetwarzanie żądań użytkownika oraz komunikację pomiędzy modelem a widokiem.

IV. Interfejs użytkownika

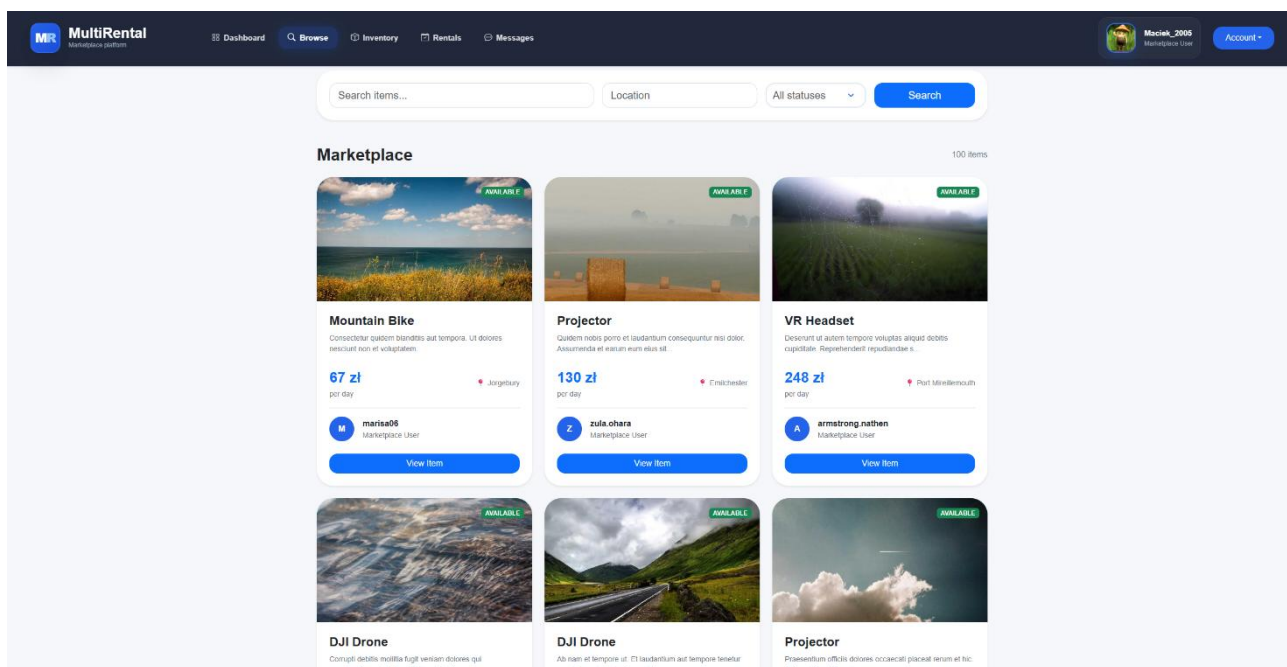
1. Opis GUI

Interfejs użytkownika aplikacji MultiRental został zaprojektowany z naciskiem na czytelność, intuicyjność obsługi oraz szybki dostęp do najważniejszych funkcjonalności systemu. Aplikacja wykorzystuje nowoczesny układ komponentów, umożliwiający sprawną nawigację pomiędzy modułami platformy.



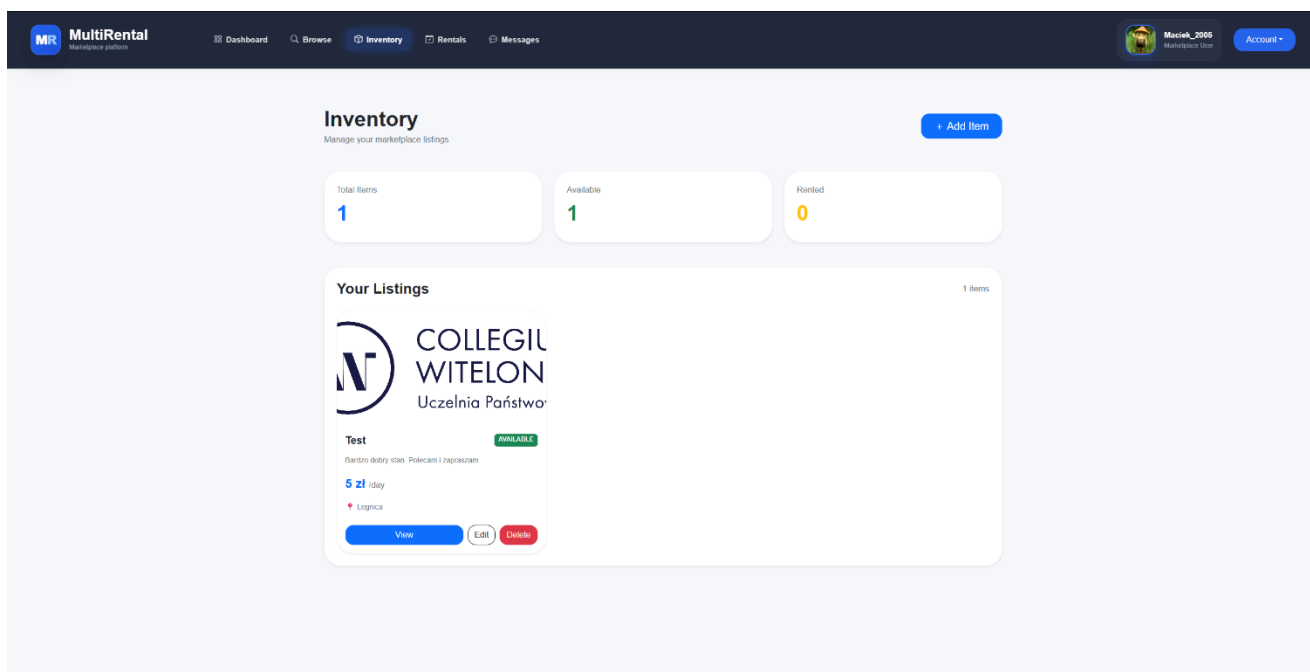
Rysunek 1. Widok dashboard

Dashboard stanowi główny ekran aplikacji po zalogowaniu użytkownika. Widok prezentuje podstawowe informacje dotyczące aktywności użytkownika oraz umożliwia szybki dostęp do najważniejszych funkcji systemu. Zawiera navigation bar zapewniający szybki dostęp do najważniejszych elementów platformy, reprezentację danych użytkownika – takich jak ilość wypożyczeń, przedmiotów na wynajem itp. a także panel „szybkich akcji” i panel ostatniej aktywności, który w chwili obecnej jest placeholderem.



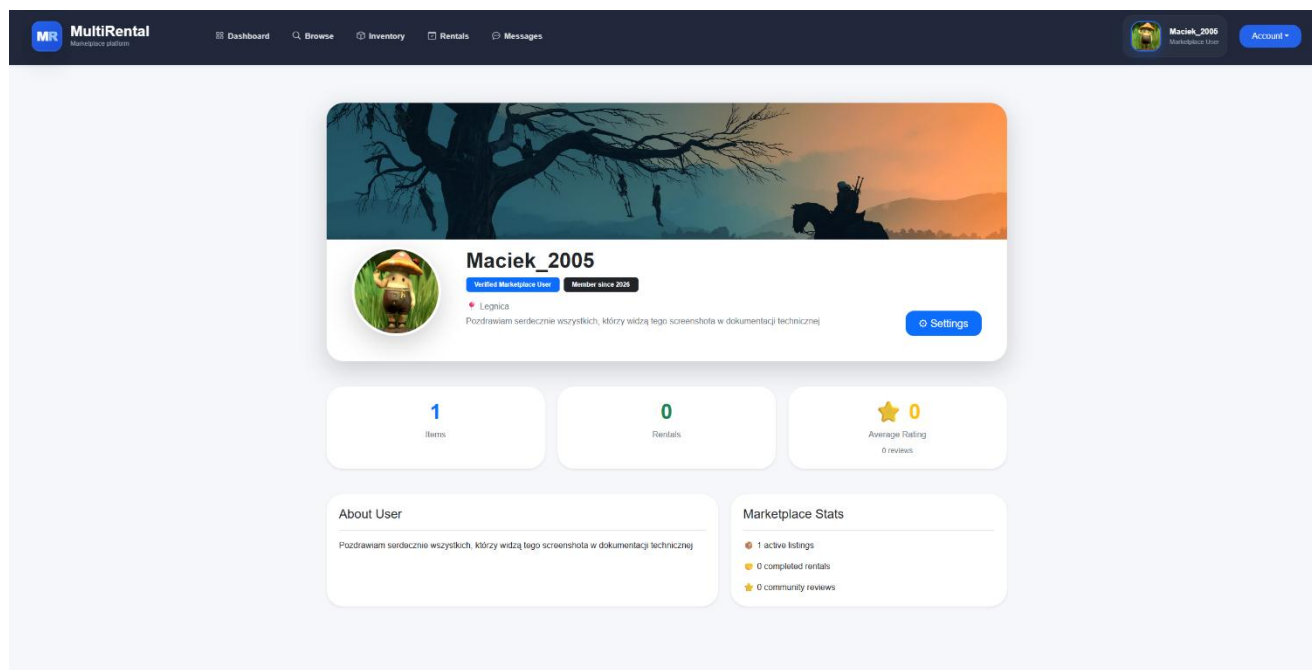
Rysunek 2. Widok marketplace (browse)

Widok marketplace umożliwia przeglądanie dostępnych ofert wynajmu przedmiotów. System pozwala na wyszukiwanie ofert oraz filtrowanie wyników według wybranych parametrów. Przedmioty posiadają zdjęcia wyświetlające się w przeglądarce – zastosowano placeholder w postaci zaimplementowania zdjęć ze strony picsum.photos do ItemFactory.



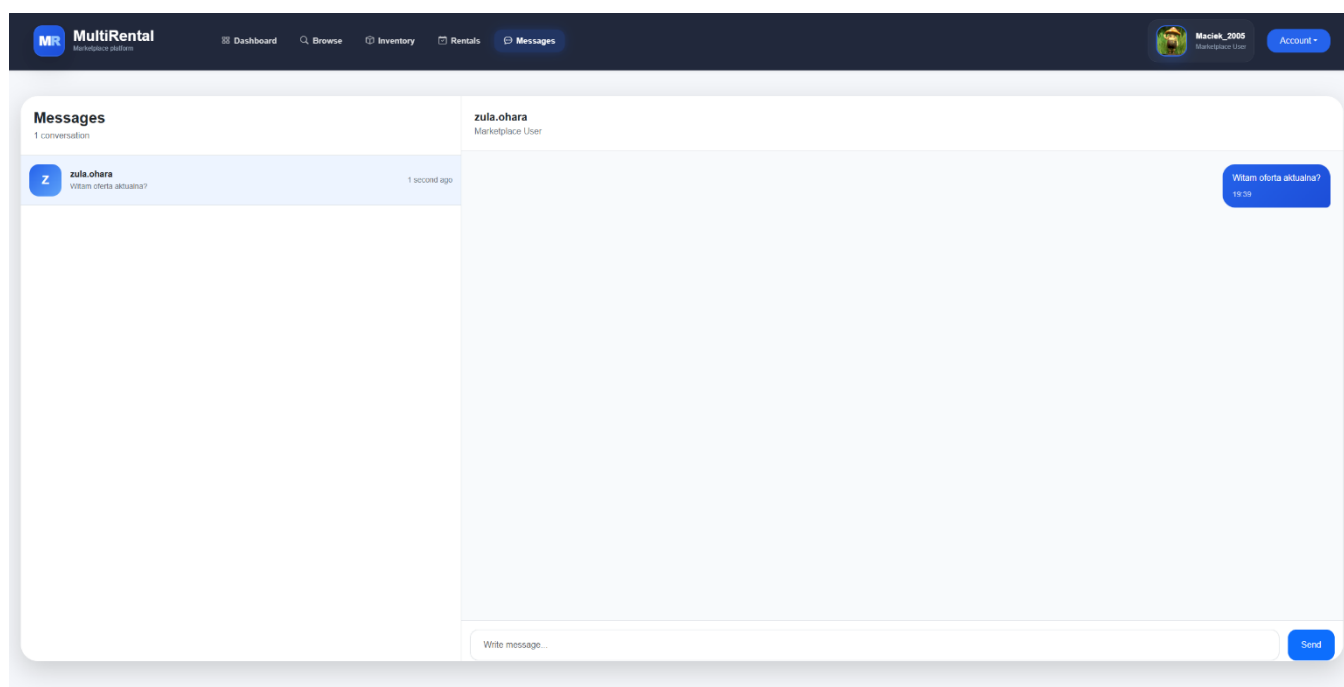
Rysunek 3. Widok inventory

Moduł inventory umożliwia zarządzanie własnymi ofertami użytkownika. Zaimplementowano pełny zakres operacji CRUD pozwalający tworzyć, edytować, usuwać oraz przeglądać własne ogłoszenia.

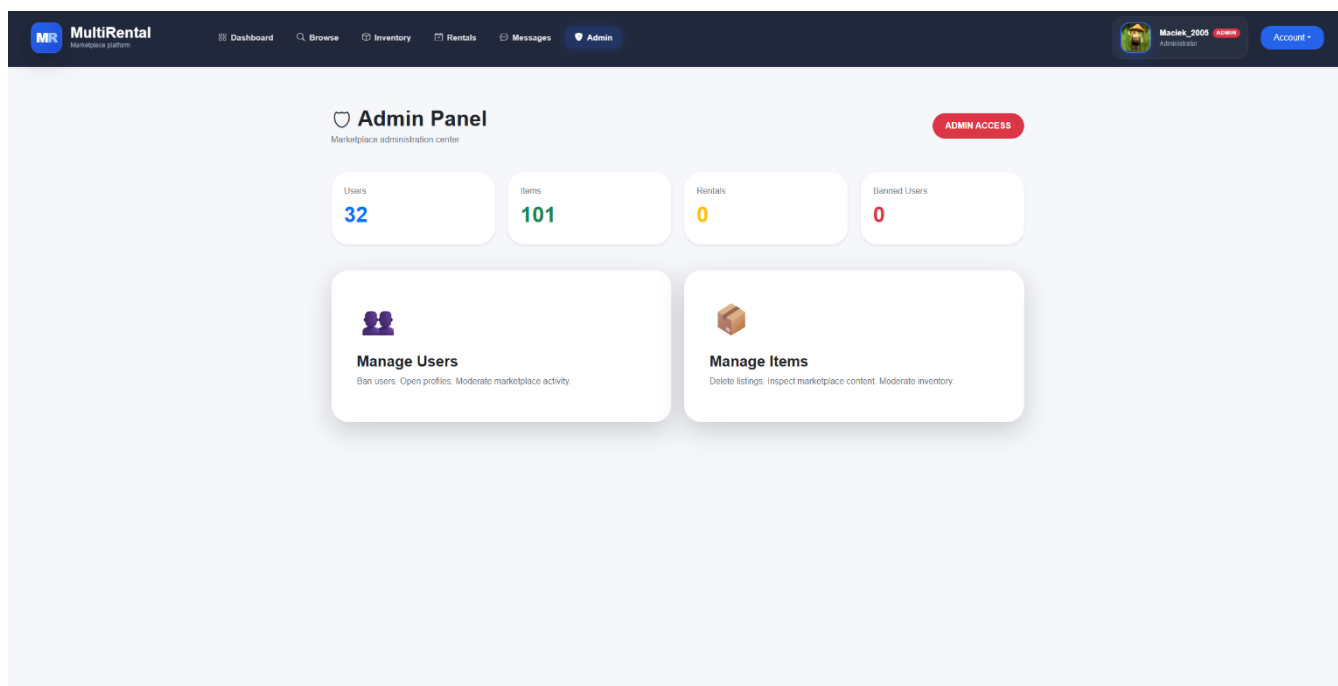


Rysunek 4. Widok profilu użytkownika

System umożliwia zarządzanie profilem użytkownika poprzez edycję podstawowych danych, dodawanie zdjęcia profilowego oraz obrazu banera.



Rysunek 5. Widok prywatnych wiadomości



Rysunek 6. Widok panelu administratora

Administrator posiada rozszerzone uprawnienia pozwalające na zarządzanie użytkownikami oraz moderację systemu.

2. Implementacja

```

1  <?php
2
3  namespace App\Http\Middleware;
4
5  use Closure;
6  use Illuminate\Http\Request;
7  use Symfony\Component\HttpFoundation\Response;
8
9  class AdminMiddleware
10 {
11     public function handle(Request $request, Closure $next): Response
12     {
13         if (!auth()->check() || !auth()->user()->is_admin) {
14             abort(403);
15         }
16
17         return $next($request);
18     }
19 }

```

Rysunek 7. Implementacja middleware administratora

Middleware odpowiada za ograniczenie dostępu do funkcjonalności administracyjnych wyłącznie dla użytkowników posiadających odpowiednie uprawnienia. W przypadku próby wejścia przez nieautoryzowanego użytkownika system zwraca kod błędu HTTP 403.

```
33     public function items()
34     {
35         return $this->hasMany(Item::class, 'owner_id');
36     }
37
38     public function rentals()
39     {
40         return $this->hasMany(Rental::class, 'renter_id');
41     }
42
43     public function reviews()
44     {
45         return $this->hasMany(Review::class);
46     }
47
48     public function reviewsReceived()
49     {
50         return $this->hasMany(Review::class, 'reviewed_user_id');
51     }
52
```

Rysunek 8. Implementacja relacji modelu użytkownika

Framework Laravel wykorzystuje mechanizm ORM Eloquent umożliwiający definiowanie relacji pomiędzy encjami systemu. Zaprezentowany fragment kodu przedstawia relacje użytkownika z przedmiotami, wypożyczeniami oraz systemem recenzji.

```
46     Message::create([
47         'conversation_id' => $conversation->id,
48         'sender_id' => Auth::id(),
49         'message' => $request->message
50     ]);
51
52     return back();
53 }
54
```

Rysunek 9. Implementacja systemu prywatnych wiadomości

Fragment kodu przedstawia proces zapisu wiadomości prywatnej do bazy danych. Każda wiadomość zostaje przypisana do konkretnej konwersacji oraz użytkownika będącego nadawcą, co umożliwia zachowanie historii komunikacji pomiędzy użytkownikami platformy.

```
1  <?php
2
3  use Illuminate\Database\Migrations\Migration;
4  use Illuminate\Database\Schema\Blueprint;
5  use Illuminate\Support\Facades\Schema;
6
7  return new class extends Migration
8  {
9      /**
10       * Run the migrations.
11       */
12  ✓ public function up(): void
13  {
14      Schema::create('rentals', function (Blueprint $table) {
15          $table->id();
16          $table->foreignId('item_id')->constrained();
17          $table->foreignId('renter_id')->constrained('users');
18          $table->date('start_date');
19          $table->date('end_date');
20          $table->decimal('total_price', 8, 2);
21          $table->enum('status', ['active', 'returned', 'cancelled']);
22          $table->timestamps();
23      });
24  }
25
26  /**
27   * Reverse the migrations.
28   */
29  public function down(): void
30  {
31      Schema::dropIfExists('rentals');
32  }
33  };
```

Rysunek 10. Implementacja migracji tabeli rentals

Fragment kodu przedstawia definicję struktury tabeli odpowiedzialnej za proces wypożyczeń. W implementacji wykorzystano mechanizm migracji frameworka Laravel umożliwiający wersjonowanie zmian struktury bazy danych oraz definiowanie kluczy obcych zapewniających integralność danych.

```

111  ✓ public function destroy(Item $item)
112      {
113          if ($item->owner_id !== Auth::id()) abort(403, 'No access.');
```

```

114
115          if ($item->image && !str_contains($item->image, 'picsum.photos')) {
116              Storage::disk('public')->delete($item->image);
117          }
118
119          $item->delete();
120          return redirect()->route('inventory')->with('success', 'Item deleted.');
```

```

121      }
122  }
```

Rysunek 11. Implementacja operacji usuwania ogłoszenia

Fragment kodu przedstawia implementację operacji DELETE w ramach mechanizmu CRUD. Przed wykonaniem usunięcia system weryfikuje uprawnienia użytkownika oraz usuwa powiązane zasoby przechowywane w pamięci aplikacji.

3. Wykorzystanie systemu kontroli wersji

W trakcie realizacji projektu wykorzystano system kontroli wersji Git oraz repozytorium GitHub umożliwiające zarządzanie kodem źródłowym, śledzenie historii zmian oraz pracę zespołową. W projekcie stosowano pracę na niezależnych gałęziach (branchach), integrację zmian przy użyciu pull requestów oraz proces przeglądu kodu przed scaleniem zmian do głównej wersji aplikacji.

Kod źródłowy projektu wraz z historią zmian dostępny jest w repozytorium:

<https://github.com/maciek-wroblewski/multi-rental-platform>

Bibliografia

Spis rysunków:

Numer rysunku	Strona	Tytuł
1	11	Widok dashboard
2	12	Widok marketplace (browse)
3	12	Widok inventory
4	13	Widok profilu użytkownika
5	13	Widok prywatnych wiadomości
6	14	Widok panelu administratora
7	14	Implementacja middleware administratora
8	15	Implementacja relacji modelu użytkownika
9	15	Implementacja systemu prywatnych wiadomości
10	16	Implementacja migracji tabeli rentals
11	17	Implementacja operacji usuwania ogłoszenia

Spis diagramów:

Numer diagramu	Strona	Tytuł
1	6	Diagram use-case
2	8	Diagram ERD